VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

WEB APPLICATION DEVELOPMENT
(IT093IU)
Instructor: Assoc. Prof. N.T.Nghia

LOCAL SPOTIFY MUSIC WEB REPORT

1.	Nguyễn Nhật Nam	ITCSIU24058	Team Leader
2.	Lương Nguyễn Tấn Dũng	ITCSIU24024	Team Member
3.	Hoàng Triệu Nam	ITCSIU24059	Team Member

1. Executive Summary

1.1. Project Name & Team:

- Local Spotify Clone

1.2. Problem Statement:

- The need for a lightweight, locally-deployed music platform that allows users to manage their personal music collections without relying on the internet

1.3. Target Users:

- Students and music lovers who want to store and play personal MP3 files, and manage their own playlists

1.4. Key Features:

- Our application delivers a production-ready music streaming experience by implementing the following core functionalities:

- Comprehensive User Security: Secure registration and login workflows utilizing the BCrypt hashing algorithm to encrypt and protect user credentials before database insertion.
- Core Media Management (File Upload & Streaming): A robust file handling system that safely processes and stores physical .mp3 files in local storage, while mapping their precise directories to a normalized relational database. The server streams these files back to the client via optimized byte arrays.
- Uninterrupted SPA Playback: A seamless Single Page Application (SPA) architecture driven by Vanilla JavaScript. This allows users to browse songs, interact with the interface, and manage playlists without triggering page reloads, ensuring the active audio stream never drops.

- Playlist Customization (Complex CRUD): Full Create, Read, Update, and Delete capabilities for users to generate personalized playlists and dynamically manage the tracks within them.
- Social Engagement & Analytics: Interactive features that mimic real-world platforms, including the ability to "Like" tracks, engage in discussions via "Comments", and an automated tracking system that displays the "Listen Count" for every uploaded song

1.5. Technology Stack Summary:

- Vanilla HTML/CSS/JS (Frontend)
- Java Spring Boot (Backend)
- MySQL (Database).

1.6. Deployment URL & Test Credentials:

- http://localhost:8080/

2. Requirements Analysis

2.1 Problem Statement:

- In the era of cloud computing, while online music streaming platforms like Spotify or Apple Music dominate the market, there is still a significant need for managing and playing locally stored audio files. Many individuals possess personal collections of MP3 files, including rare tracks, unreleased covers, podcasts, or custom audio recordings that are not available on mainstream platforms. However, playing these files usually requires relying on basic, outdated offline media players that lack modern features such as seamless web-based playback, interactive UI, and playlist management. Our project, Local Spotify Clone, solves this by providing a modern, web-based, self-hosted platform where users can upload, organize, and stream their local music library with a premium user experience similar to top-tier streaming services, all without requiring a paid subscription or internet reliance.
- The primary users of this application are:
    - Students and Young Adults: Individuals who want a free, customizable platform to listen to music while studying or working without being interrupted by ads.
    - Music Collectors & Enthusiasts: Users who have large local collections of MP3s (e.g., lossless rips, indie music,

non-copyrighted tracks) and want a unified, aesthetic interface to manage them
    - Small Communities/Local Networks: Groups of friends or colleagues sharing a local server who want to discover and interact with each other's music tastes.
- Before using our system, these users typically face the following frustrations:
    - Scattered Media Files: MP3 files are often disorganized in random computer folders, making it difficult to search, filter, or group them into specific moods or playlists.
    - Lack of Modern UI/UX: Traditional desktop media players have clunky interfaces. Users want the sleek "Dark Mode" aesthetic and smooth Single Page Application (SPA) experience they are used to on modern web apps
    - No Social Interaction: Offline players isolate the listening experience. Users lack a way to express their feelings about a track (via Likes) or discuss it with others (via Comments) within their local network
    - Device Dependency: To listen to downloaded music, users usually have to transfer files manually between devices via USB or cables. A local web-based solution allows them to access their music from any browser on the same network

2.2 User Stories

- Based on the system requirements and feature scope of the Local Spotify Clone, our team has defined the following user stories. They are categorized by priority (Must Have, Should Have, Nice to Have) to guide the development phases
- Priority 1: Must Have (Core Functionality)
    - As a Guest, I want to register a new account using a username and password, so that I can become a member of the platform.
    - As a User, I want to log in securely to the system, so that I can access my music library and personal data.
    - As a User, I want to upload an MP3 audio file from my local machine, so that I can contribute songs to the shared library.
    - As a User, I want to click the Play button to stream a song directly in the browser, so that I can enjoy the music.

    - As a User, I want to keep the music playing without interruption when navigating between pages (SPA behavior), so that my listening experience is seamless.
    - As a User, I want to view a list of all uploaded songs on the main interface, so that I can select a track to listen to.
- Priority 2: Should Have (Important Interactions)
    - As a User, I want to create a new Playlist and name it, so that I can organize my music by mood or preference
    - As a User, I want to add any song to my personal Playlist, so that I can easily find and replay it later
    - As a User, I want to view all my Playlists on the sidebar, so that I can quickly switch between my collections
    - As a User, I want to 'Like' a song, so that I can show my appreciation for that track.
    - As a User, I want to write a comment under a song, so that I can share my thoughts with other users
    - As an Uploader, I want to see the system automatically count and display the listen count, so that I know the popularity of the songs I uploaded.
- Priority 3: Nice to Have (UX/UI Enhancements)
    - As a User, I want to see the currently playing song's title and artist on the bottom player, so that I always know what track is playing
    - As a User, I want to be able to remove a song from my Playlist, so that I can keep my collections updated
    - As a User, I want to log out of my account securely, so that my account cannot be accessed by others using the same device

2.3 Functional Requirements:

Functional requirements define the specific behaviors, actions, and features the system must support. To ensure clarity, they are categorized based on user roles and core modules.

- Guest Role (Unauthenticated User)
    - FR-1.1 Registration: The system must allow guests to create a new account by providing a unique username and a password.

- FR-1.2 Authentication: The system must allow guests to log in using their registered credentials to access the main platform.

- Authenticated User Role
  - Music Streaming & Management:
    - FR-2.1 Upload: Users must be able to upload audio files (specifically .mp3 format) from their local device to the server.
    - FR-2.2 Browse: Users must be able to view a comprehensive list of all uploaded songs available on the platform.
    - FR-2.3 Playback: Users must be able to play, pause, and control the volume of a selected track via an integrated media player.
    - FR-2.4 Uninterrupted Listening: The audio player must continue playing seamlessly without resetting or stopping when the user navigates between different views (e.g., switching from Home to a Playlist).
- Playlist Management:
    - FR-2.5 Create Playlist: Users must be able to create new custom playlists and assign names to them.
    - FR-2.6 Modify Playlist: Users must be able to add existing songs to their playlists or remove them.
    - FR-2.7 View Playlists: Users must be able to retrieve and view the content of their specific playlists.
- Social Interactions:
    - FR-2.8 Like/Unlike: Users must be able to toggle a "Like" status on any song.
    - FR-2.9 Commenting: Users must be able to post text comments on specific songs to share their thoughts.
- Account Management:
    - FR-2.10 Logout: Users must be able to securely terminate their active session and log out of the system.
- System Behaviors (Automated)
  - FR-3.1 Listen Count Tracking: The system must automatically increment a song's "listen count" metric in the database each time the track is successfully requested and streamed by a user.

2.4 Non-Functional Requirements:

Non-functional requirements specify the system's quality attributes, performance constraints, and architectural standards.

- Security Requirements
    - Password Cryptography: All user passwords must be hashed using the strong BCrypt algorithm before being persisted to the MySQL database. Plaintext passwords must never be stored or exposed in logs.
    - Endpoint Protection: All RESTful APIs related to uploading, streaming, and interacting with music must be secured. Only authenticated users can access these endpoints.
    - Data Sanitization: The system (via Spring Data JPA) must utilize prepared statements and ORM mappings to prevent SQL Injection attacks.
- Performance & Efficiency
    - Audio Streaming Optimization: The backend server (Spring Boot) must process and serve audio files as byte streams rather than loading entire files into memory. This ensures fast buffering and prevents server crashes when handling large .mp3 files.
    - Dynamic Rendering (SPA): The frontend must utilize asynchronous JavaScript (Fetch API) to dynamically update content sections (DOM manipulation) without triggering full page reloads, preserving the state of the bottom audio player.
- Usability and User Experience (UX)
    - Aesthetic Design: The application must implement a modern, high-contrast "Dark Mode" user interface (UI) to reduce eye strain, mimicking industry-standard streaming platforms.
    - System Feedback: The UI must provide immediate, clear feedback for user interactions, such as error alerts for failed logins or visual confirmation (e.g., color changes) when a song is successfully added to a playlist.
- Maintainability and Code Quality
    - Layered Architecture: The backend infrastructure must strictly adhere to the Controller-Service-Repository design pattern to ensure a clean separation of concerns and facilitate independent workflows for a 3-member development team.
    - Database Normalization: The relational database schema must be normalized to at least the Third Normal Form (3NF) to guarantee data

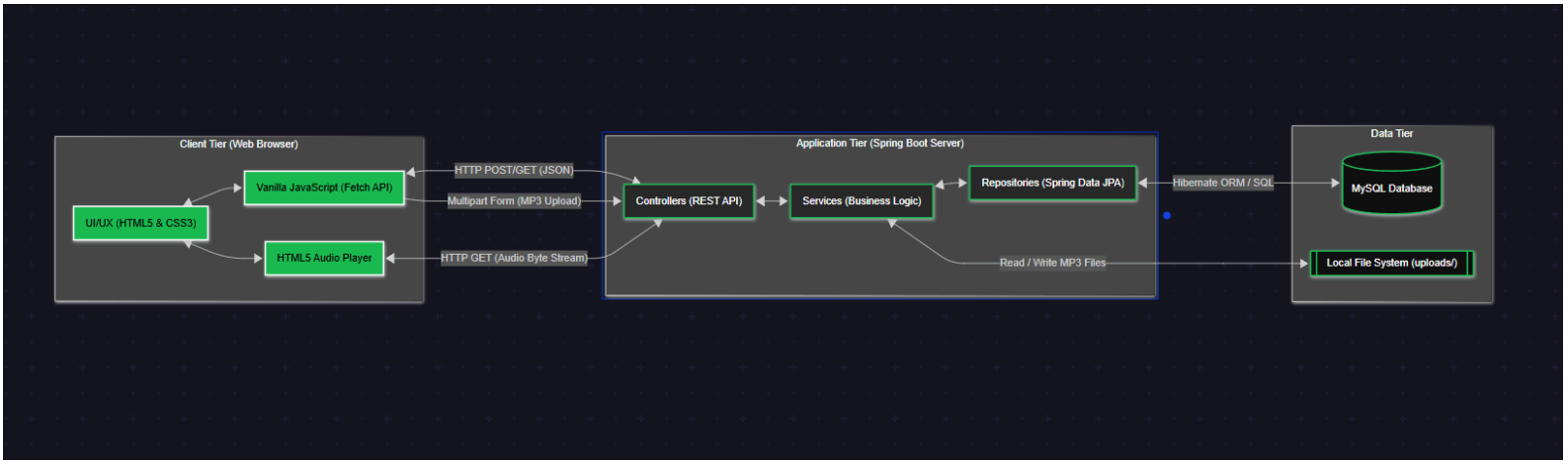
integrity and eliminate redundancies (e.g., separating user interactions into dedicated likes and comments tables).

3. System Architecture

3.1 Architecture Overview:

The Local Spotify Clone follows a modern Client-Server-Database architectural model, operating as a Single Page Application (SPA) communicating with a RESTful API backend.

Data Flow Explanation:



1. Client Tier (Frontend): The user interacts with the browser-based UI. The Vanilla JavaScript layer captures these events and sends asynchronous HTTP requests (via Fetch API) to the server.
2. Application Tier (Backend): The Java Spring Boot server intercepts the requests via Controllers. The Controllers delegate business logic and data processing to the Service layer, which then interacts with the Repository layer for database operations. For audio files, the server reads the .mp3 physical files from the local uploads/ directory and returns them as a byte stream.
3. Data Tier (Database): The MySQL database executes the queries (mapped via Hibernate ORM) to retrieve or modify structured relational data (users, playlists, interactions) and returns the result set back to the Application tier.

3.2 Technology Stack Justification:

- Choosing the right technology stack was critical to balancing project complexity, team expertise, and performance requirements:

- Frontend: JavaScript, HTML5, CSS3 vs. Modern Frameworks (React/Vue)
    - Our Choice: Vanilla JavaScript.
    - Justification: While frameworks like React or Vue are excellent for complex states, we chose Vanilla JavaScript to demonstrate a deep, fundamental understanding of DOM manipulation, asynchronous programming (Fetch API), and event handling. For a music player SPA where the primary challenge is maintaining the audio state across virtual page transitions, Vanilla JS provides full control without the overhead of a virtual DOM, ensuring a lightweight and highly performant client-side application.
- Backend: Java Spring Boot vs. Java Servlets/JSP
    - Our Choice: Java Spring Boot (3.x).
    - Justification: We chose Spring Boot over raw Java Servlets because it is an industry-standard framework that significantly boosts productivity. Raw Servlets require extensive boilerplate code for routing, JSON serialization, and database connections. Spring Boot provides built-in modules like Spring Web (for rapid REST API development), Spring Data JPA (for robust ORM), and Spring Security (for BCrypt integration), allowing the team to focus on complex business logic (like audio streaming) rather than infrastructure setup

- Database: MySQL vs. NoSQL (MongoDB)
    - Our Choice: MySQL (Relational Database).
    - Justification: The core data of our application (Users, Songs, Playlists, Comments, Likes) is highly relational. A user can have many playlists (1-to-N), and a playlist can contain many songs (M-to-N). We chose MySQL over a document-based database like MongoDB because a relational schema enforces strict data integrity through foreign keys and cascading operations. Normalizing this data (to 3NF) in MySQL prevents anomalies and ensures accurate

aggregation queries, such as counting total likes or streams for a specific song

3.3 Design Patterns:

To ensure a clean, maintainable, and scalable codebase, our team implemented several well-established software design patterns:

- Layered Architecture Pattern: The backend is strictly divided into three layers: Controller (handles HTTP requests and responses), Service (encapsulates business logic), and Repository (manages database access). This separation of concerns prevents spaghetti code and allows team members to work independently.
- Data Transfer Object (DTO) Pattern: We utilized DTOs (e.g., AuthDto, ApiResponse) to pass data between the client and server. This ensures that internal database entities (like the User class containing password hashes) are never directly exposed to the frontend, enhancing security.
- Singleton Pattern: Managed implicitly by the Spring IoC (Inversion of Control) container, all our Service and Repository classes are instantiated as Singletons, optimizing memory usage and ensuring stateless execution across multiple API requests.
- Singleton Page Application (SPA) Pattern: On the frontend, instead of traditional MVC where the server renders HTML, we implemented an API-driven SPA. The index.html file acts as the single entry point, dynamically rendering specific sections while preserving the state of the global audio player

3.4 API Design:

The backend communication follows strict RESTful API principles to ensure consistency and predictability for the frontend consumers.

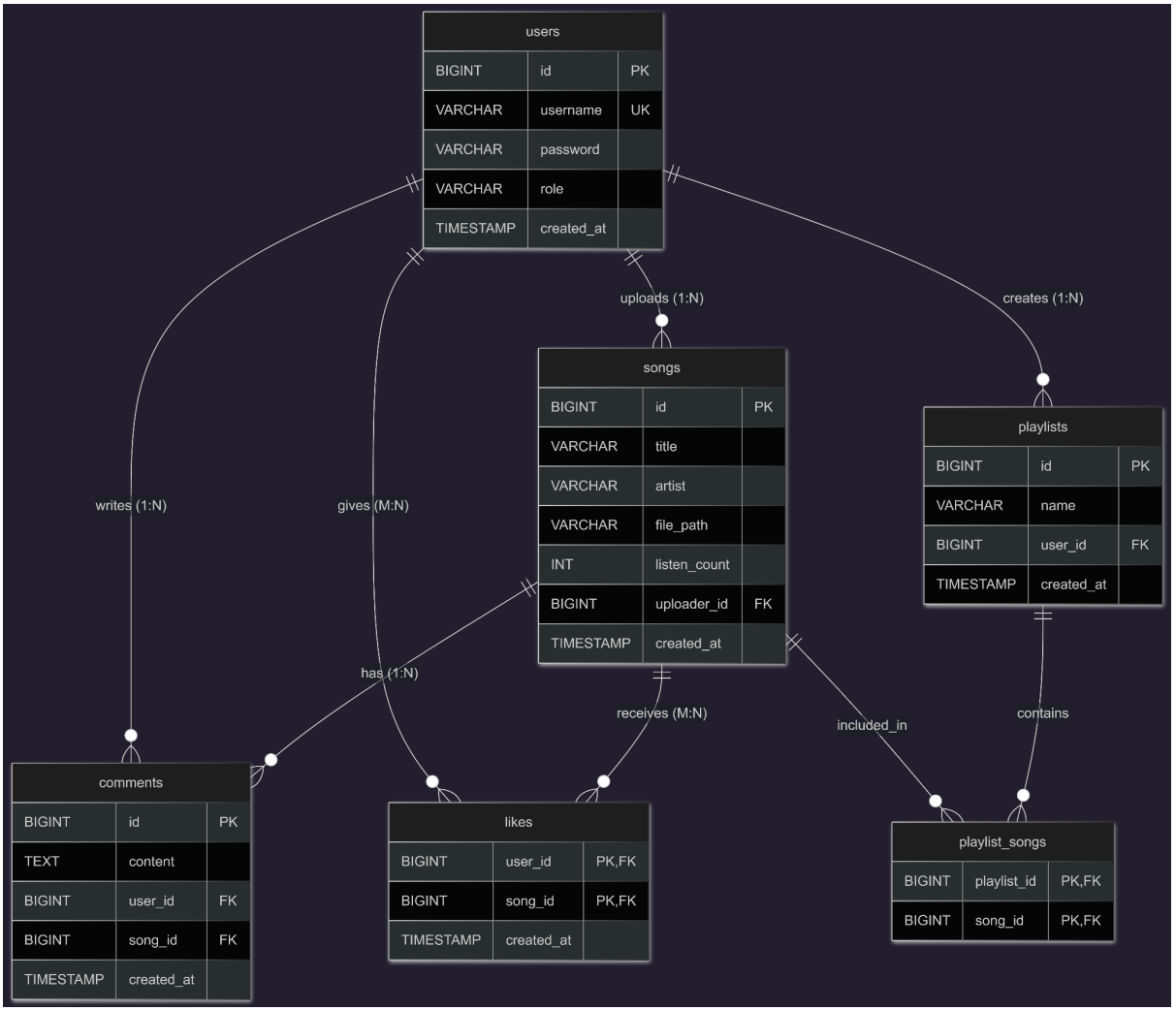
- URL Structure: All endpoints utilize resource-based routing with a versioning prefix (e.g., /api/v1/...).
- HTTP Methods: We mapped CRUD operations to standard HTTP methods:
    - GET: To retrieve resources (e.g., GET /api/songs to fetch the song list).

- POST: To create new resources (e.g., POST /api/auth/register, POST /api/playlists).
- PUT/DELETE: To modify or remove resources (e.g., DELETE /api/playlists/{id}).

- Standardized Responses & Status Codes: Every API response is wrapped in a generic ApiResponse<T> structure containing a message, data payload (in JSON format), and appropriate HTTP status codes:
  - 200 OK / 201 Created for successful operations.
  - 400 Bad Request for validation failures (e.g., username already exists).
  - 401 Unauthorized / 403 Forbidden for security rejections.
  - 500 Internal Server Error for unhandled backend exceptions.

4. Database Design

4.1 Entity-Relationship Diagram:



The ER diagram illustrates the logical structure of the Local Spotify Clone database. It explicitly defines the relationships between 6 entities: users, songs, playlists, playlist_songs (junction table), likes (junction table), and comments. Crow's foot notation is used to represent cardinalities clearly.

4.2 Table Schemas:

The database consists of 6 tables, utilizing surrogate keys (BIGINT with AUTO_INCREMENT) for primary identification to ensure scalability.

Table: users

- id: PRIMARY KEY AUTO_INCREMENT
- username: VARCHAR(50) UNIQUE NOT NULL
- password: VARCHAR(255) NOT NULL
- role: VARCHAR(20) DEFAULT 'USER'
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table: songs

- id: PRIMARY KEY AUTO_INCREMENT
- title: VARCHAR(255) NOT NULL
- artist: VARCHAR(255) DEFAULT 'Unknown Artist'
- file_path: VARCHAR(500) NOT NULL
- listen_count: INT DEFAULT 0
- uploader_id: BIGINT, FOREIGN KEY REFERENCES users(id) ON DELETE SET NULL
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table: playlists

- id: BIGINT PRIMARY KEY AUTO_INCREMENT
- name: VARCHAR(255) NOT NULL
- user_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table: playlist_songs (Junction Table)

- playlist_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES playlists(id) ON DELETE CASCADE
- song_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES songs(id) ON DELETE CASCADE
- *Constraint: PRIMARY KEY (playlist_id, song_id)*

Table: likes (Junction Table)

- user_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE

- song_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES songs(id) ON DELETE CASCADE
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- *Constraint: PRIMARY KEY (user_id, song_id)*

Table: comments

- id: BIGINT PRIMARY KEY AUTO_INCREMENT
- content: TEXT NOT NULL
- user_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE
- song_id: BIGINT NOT NULL, FOREIGN KEY REFERENCES songs(id) ON DELETE CASCADE
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

4.3 Relationships Explanation:

Our domain model accurately maps complex data relationships using foreign key constraints at the database level:

- One-to-Many (1:N):
    - User to Playlists/Comments: A single user can create multiple playlists and write multiple comments. We implemented ON DELETE CASCADE here. If a user is removed, all their associated playlists and comments are automatically purged to prevent orphan data.
    - User to Songs (Uploader): A user can upload many songs. However, we used ON DELETE SET NULL for the uploader_id foreign key. If a user is deleted, their uploaded songs remain in the global library (with a null uploader) so that other users' playlists are not abruptly broken.
- Many-to-Many (M:N):
    - Playlists and Songs: A playlist contains many songs, and a song can belong to many playlists. We resolved this using the playlist_songs junction table.
    - Users and Liked Songs: A user can like many songs, and a song can be liked by multiple users. This is handled by the likes junction table.

4.4 Normalization Analysis:

The database schema has been designed to satisfy the Third Normal Form (3NF) to ensure data integrity and eliminate redundancy:

- First Normal Form (1NF): All attributes contain atomic values. There are no repeating groups (e.g., instead of storing an array of song IDs in the playlists table, we created the playlist_songs table).
- Second Normal Form (2NF): All tables have a single-column primary key (surrogate ID), or a composite primary key (likes, playlist_songs) where the non-key attributes (like created_at in likes) depend on the entire composite key.
- Third Normal Form (3NF): There are no transitive dependencies. Every non-key attribute depends strictly and only on the primary key.
- Justified Denormalization: The artist column in the songs table is stored as a simple VARCHAR string instead of creating a separate artists table. This denormalization is justified because our scope focuses on local user uploads (where metadata is often inconsistent) rather than maintaining a strict global artist metadata directory. It significantly reduces JOIN complexity during audio fetching.

4.5 Indexing Strategy:

To optimize query performance, we implemented the following indexing strategy:

- Primary Key Indexes: Automatically created by MySQL on all id columns (Clustered Index), ensuring $O(\log n)$ retrieval time for exact matches.
- Foreign Key Indexes: Indexes are implicitly/explicitly created on all foreign keys (user_id, song_id, playlist_id, uploader_id) to accelerate JOIN operations and enforce referential integrity checks efficiently.
- Unique Constraint Index: The username column in the users table has a UNIQUE index. This drastically improves the performance of the login authentication process, as the system frequently queries SELECT * FROM users WHERE username = ?

5. Implementation Details

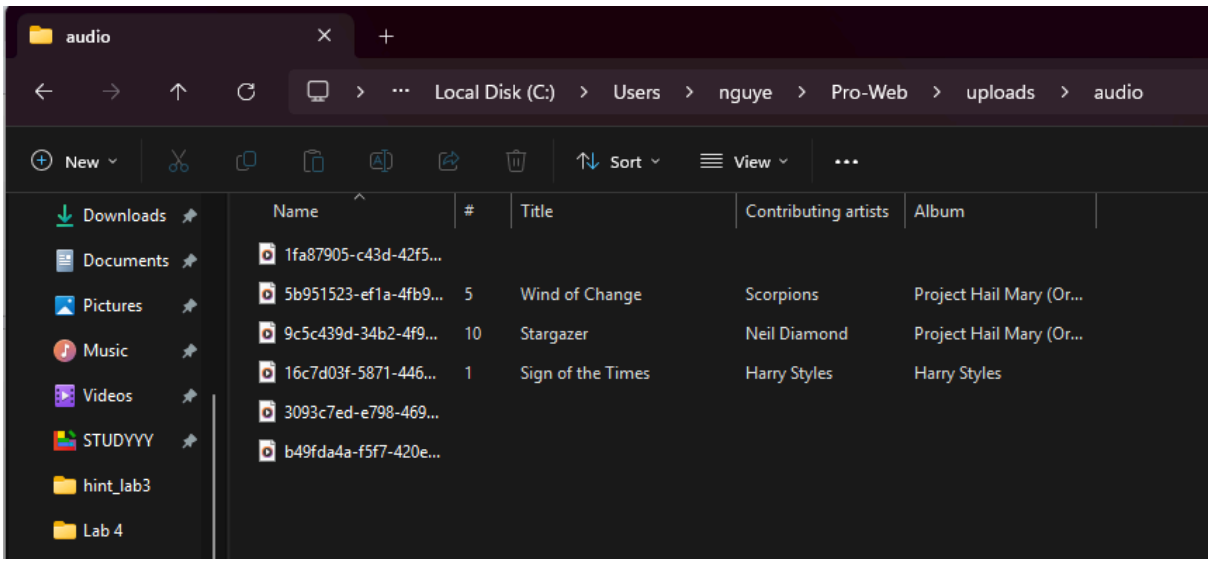
5.1 Key Algorithms and Business Logic:

- Audio Streaming Algorithm (Byte Array Stream) Instead of loading the entire audio file (which can be tens of megabytes) into the server's RAM and sending it all at once, the system utilizes a chunked streaming technique.

- **Logic:** The application reads the physical MP3 file from the local disk and wraps it in a FileSystemResource object. When returning the ResponseEntity, the MediaType is explicitly set to APPLICATION_OCTET_STREAM. Spring Boot automatically processes this by breaking the byte array into smaller chunks and streaming them over the network. The client's browser receives these bytes, buffers them, and plays the audio instantly without waiting for the full download.The songs will be read from uploads/audio

```
111     @GetMapping("/{id}/stream")
112     public ResponseEntity<Resource> streamSong(@PathVariable Long id) {
113         try {
114             Song song = songService.getSongById(id);
115             if (song == null) {
116                 return ResponseEntity.notFound().build();
117             }
118
119             File audioFile = new File(song.getFilePath());
120             if (!audioFile.exists()) {
121                 return ResponseEntity.notFound().build();
122             }
123
124             // Tăng số lần nghe
125             songService.incrementListenCount(id);
126
127             Resource resource = new FileSystemResource(audioFile);
128             return ResponseEntity.ok()
129                     .contentType(MediaType.APPLICATION_OCTET_STREAM)
130                     .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + song.getTitle() + ".mp3\"")
131                     .body(resource);
132
133         } catch (Exception e) {
134             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
135         }
136     }
137
```

5.1.1. Audio Streaming Algorithm



5.1.2. Place to save the MP3 Files

Seamless SPA Navigation Logic In a traditional multi-page application, navigating from the "Home" page to the "Library" page would reload the browser, terminating the <audio> tag and stopping the music.

- **Logic:** The frontend is engineered as a Single Page Application (SPA). All page contents are pre-loaded within <section> tags. When a user clicks the navigation menu, a JavaScript function (showView) dynamically toggles the CSS display property (block vs. none) to hide and reveal sections. The persistent <audio> player is positioned in a fixed bottom-player container outside of these sections, guaranteeing that the audio stream remains uninterrupted during UI navigation.

```
71  function showView(viewId) {
72      const views = ['home','search','library','liked','upload'];
73      views.forEach(v => {
74          const el = document.getElementById('view-' + v);
75          if (!el) return;
76          el.style.display = (v === viewId) ? 'block' : 'none';
77      });
78
79      if (viewId === 'liked' && currentUser) {
80          loadLikedSongs(currentUser.id);
81      }
82  }
83
```

5.1.3. Function showView

5.2 Security Implementation:

Security is a non-negotiable aspect of our application, implemented at multiple layers:

- Password Security: Plaintext passwords are never stored. We utilized Spring Security's BCryptPasswordEncoder to hash user passwords with a secure salt before saving them to the database. Even if the database is compromised, the hashes cannot be reversed.
- SQL Injection Prevention: By strictly utilizing Spring Data JPA (Hibernate) and its method name query derivation (e.g., findByUsername), all database queries are automatically parameterized. We explicitly avoided any manual string concatenation for SQL queries.
- XSS Prevention: In the frontend, any user-generated content (such as Comments or Playlist names) is inserted into the DOM using textContent rather than innerHTML, effectively neutralizing Cross-Site Scripting (XSS) payload executions.
- CORS Configuration: We implemented a global WebMvcConfigurer to explicitly control Cross-Origin Resource Sharing (CORS), ensuring our backend only accepts requests from trusted frontend origins while allowing necessary HTTP methods and credentials.

5.3 Error Handling Strategy:

- The system implements a comprehensive, end-to-end error handling architecture. This ensures that any runtime anomalies originating from the database or the business service layer are gracefully intercepted, structured, and passed back to the frontend without exposing vulnerable server-side diagnostics (such as raw stack traces).

5.3.1. Centralized Backend Error Handling via GlobalExceptionHandler

- On the Spring Boot server, the team deployed a centralized exception handling component annotated with @RestControllerAdvice. This component functions as a global interceptor, automatically capturing thrown exceptions from any active controller node before they propagate back to the HTTP client.
- Instead of allowing the server to render default, uninformative web container failure containers (such as the standard Tomcat White Label Error Page), the GlobalExceptionHandler translates Java exceptions into a unified JSON format using the predefined ApiResponse<T> wrapper class (consisting of status, message, and a nullified data payload).
  package com.localspotify.exception;

Coding:

```
import com.localspotify.dto.ApiResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(Exception.class)
   public ResponseEntity<ApiResponse<Object>>
handleGeneralException(Exception ex) {
      ApiResponse<Object> response = new ApiResponse<>(
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "Đã xảy ra lỗi hệ thống: " + ex.getMessage(),
            null
      );
      return new ResponseEntity<>(response,
HttpStatus.INTERNAL_SERVER_ERROR);
   }
}
```

Operational Pipeline: When the service layer detects a duplicate registration conflict, it triggers a throw new IllegalArgumentException("Username is already taken!"); statement. The GlobalExceptionHandler hooks this event, transforms it into an optimized JSON entity { "status": 400, "message": "Username is already taken!", "data": null }, and dispatches it over the network with a standard HTTP 400 Bad Request status code.

5.3.2. End-to-End Error Synchronization Workflow

1. **User Action Fault:** A user submits faulty input (e.g., providing an incorrect login password string).
2. **Payload Dispatch:** The frontend sends an asynchronous JSON object across the pipeline via the native fetch() interface.
3. **Backend Exception Trigger:** The backend business service discovers a mismatch during profile evaluation and fires an internal exception: throw new IllegalArgumentException("Incorrect password!");.

4. **Centralized Interception:** The GlobalExceptionHandler intercepts the runtime exception, builds an instance of ApiResponse<Void>, and packages it into a JSON message with status: 400 and message: "Incorrect password!".
5. **UI Notification:** The try...catch node inside the client-side JavaScript receives the response stream, validates that response.ok resolves to false, parses the custom payload, and triggers the target browser alert box: alert("[Error 400]: Incorrect password!");.

5.4 Advanced Features Implementation:

5.4.1. Core Media Management: File Upload & Streaming Architecture

The multimedia processing layer of the **Local Spotify** project utilizes physical disk writing (Local File System System) mirrored with database logical references to save RAM overhead and keep application state decoupled.

- **File Upload Execution Flow**

When a client uploads an audio tệp file (.mp3), the engine processes it via the following steps:

1. **Frontend Client:** Packages binary stream bytes from an <input type="file"> node into a native JavaScript FormData instance along with text metadata keys (title, artist). Fires an asynchronous POST request with a multipart/form-data encoding header.
2. **Backend Controller:** Intercepts the request and binds the audio payload into a Spring Web MultipartFile interface.
3. **Backend Service:** * Asserts file validity (enforces validation check for audio/mpeg content-type or .mp3 naming structure extensions).
    - Appends an active timestamp token prefix System.currentTimeMillis() to the filename to guarantee global file system uniqueness and avoid collisions.
    - Leverages Files.copy() to write the raw byte stream directly to the target directory folder on the server disk storage (e.g., C:/uploads/audio/).
    - Maps the storage path to the filePath property of a Song entity and triggers songRepository.save(song) to insert the metadata row into MySQL.

```java
public Song uploadSong(MultipartFile file, String title, String artist, User uploadedBy) throws IOException {
    // Tạo thư mục upload nếu chưa tồn tại
    Path uploadPath = Paths.get(uploadDir);
    if (!Files.exists(uploadPath)) {
        Files.createDirectories(uploadPath);
    }

    // Tạo tên tệp duy nhất
    String filename = UUID.randomUUID() + "_" + file.getOriginalFilename();
    Path filePath = uploadPath.resolve(filename);       namnhvt, 3 days ago • Thêm …

    // Lưu tệp
    Files.copy(file.getInputStream(), filePath);

    // Tạo thực thể Song
    Song song = new Song();
    song.setTitle(title != null && !title.trim().isEmpty() ? title : file.getOriginalFilename());
    song.setArtist(artist != null && !artist.trim().isEmpty() ? artist : "Unknown Artist");
    song.setFilePath(filePath.toString());
    song.setFileSize(file.getSize());
    song.setUploadedBy(uploadedBy);
    song.setIsPublic(isPublic: true);

    return songRepository.save(song);
}
```

- **Audio Content Streaming Pipeline**

To ensure high-performance performance and smooth playback, the server drops full-file downloads in favor of Partial Content/Chunked Audio Streaming using HTTP ResourceRegion configurations:

1. **Partial Content Request:** The browser natively requests audio fragments via the HTML5 <audio> player, embedding an HTTP header parameter matching Range: bytes=0-.
2. **Server-Side File Resolution:** The endpoint maps the path string via FileSystemResource. Using the requested Range header, the server isolates parts of the file into specific byte segments (typically sized at 1MB chunks).
3. **HTTP 206 Partial Content Response:** The server responds with an HTTP status code 206, letting the browser client parse chunk streams iteratively. This allows instantaneous seeking (scrubbing through the track progress bar) without forcing the application client to cache the entire audio block.

5.4.2. Advanced Playlist Data Management & Interactions

As the volume of tracks, user-made playlists, and persistent reactions (likes, comments) grows, specific architectural mitigations are integrated to maintain stable sub-200ms query speeds and prevent bottlenecks.

- **Many-to-Many Association Optimization**

The tables handling track compilation within playlists (playlist_songs) and tracking song preferences (likes) form heavy Many-to-Many relationships. To avoid recursive memory loading and mitigate the N+1 Query Problem, the following JPA designs are enforced:

- **Enforcing FetchType.LAZY:** Relational bindings (@ManyToMany, @OneToMany) default to lazy fetching. Connected datasets (e.g., comments linked to a track row) are never loaded into JVM memory until the frontend explicitly requests them via an active accessor method like getComments().
- **Decoupled Data Transfer Objects (DTO):** Outbound payloads bypass raw DB Entity models. Sensitive or circular values are omitted via intermediary transfer schemas (AuthDto, SongDto), minimizing packet size over the local network and speeding up client UI rendering.

- **High-Volume Indexing & Data Integrity Strategies**

To keep lookups fast even under high data volume scenarios, specific database indexing methods are applied at the SQL level:

- **Composite Primary Keys:** Intermediary bridge tables (playlist_songs, likes) bind foreign keys together into a dual-column Clustered Index primary key. This ensures that checking whether a user has liked a track happens at an optimized algorithmic efficiency of $O(1)$.
- **Cascading Foreign Key Constraints:** Relational linkages specify ON DELETE CASCADE rules. When a user account or a playlist is removed, child rows in the likes, comments, and junction tables are wiped instantly by the database engine, preventing data orphans and protecting storage space from fragmentation.

5.5 AI Assistance Report:

Throughout the development of the "Local Spotify" web application, the development team leveraged Artificial Intelligence tools, primarily **ChatGPT (OpenAI), Gemini** for architectural brainstorming and problem-solving, and **GitHub Copilot** for syntax completion and boilerplate generation.

While AI significantly accelerated the initial coding phases—especially for standard RESTful APIs and UI layouts—the team recognized that AI-generated code is often generic, sometimes insecure, and lacks context regarding the specific architecture of the project. Therefore, a strict policy was adopted: all AI-generated code must be critically reviewed, refactored for security, and adapted to fit the project's specific business logic (such as Local File System storage and custom CORS policies).

Below are major case studies detailing how AI was utilized, the initial output it provided, and the specific engineering improvements made by the team.

Case Study 1: Database Schema Generation and Data Normalization

Context: At the beginning of the project, the team needed a foundational MySQL relational schema to handle Users, Songs, and interactions (Likes, Comments, Playlists).

Prompt Used:

> *"Write a comprehensive MySQL database script for a music streaming application. It needs tables for Users and Songs. Ensure there are foreign keys connecting the song to the user who uploaded it."*

Raw AI-Generated Code:

```
SQL
CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50) NOT NULL UNIQUE,
   password VARCHAR(255) NOT NULL
);

CREATE TABLE songs (
   id INT AUTO_INCREMENT PRIMARY KEY,
   title VARCHAR(100) NOT NULL,
   artist VARCHAR(100),
   file_path VARCHAR(255) NOT NULL,
   uploader_id INT,
   FOREIGN KEY (uploader_id) REFERENCES users(id)
);
```

Team Modifications & Improvements: The AI provided a functional but overly simplistic schema. The team made the following critical improvements to meet the project's actual requirements:

1. **Data Normalization & Business Logic:** The AI missed several crucial fields required for a real music player. We manually expanded the songs table to include album, duration, file_size (for frontend validation), and a boolean is_public flag to allow users to keep tracks private.
2. **ORM Synchronization (Spring Data JPA):** The AI named the foreign key uploader_id. However, in our Spring Boot @Entity class (Song.java), we defined the relationship using @JoinColumn(name = "uploaded_by"). To prevent Hibernate 500 Internal Server Errors caused by mismatched columns, we refactored the database schema.
3. **Automated Schema Management:** Instead of relying entirely on static AI SQL scripts, the team implemented spring.jpa.hibernate.ddl-auto=update in application.properties. This allowed our Java Entities to act as the single source of truth, dynamically patching the database schema during runtime.

3. Case Study 2: Cross-Origin Resource Sharing (CORS) Security

Context: When connecting the static HTML/JS frontend to the Spring Boot backend, the browser blocked the login requests due to CORS policies. The frontend needed to send authentication credentials, which triggered strict browser security rules.

Prompt Used:

> *"I am building a Spring Boot backend and a vanilla JS frontend. I am getting a CORS error: 'When allowCredentials is true, allowedOrigins cannot contain the special value *'. *How do I fix this in my AuthController?"*

Raw AI-Generated Code:

Java

```
@RestController
@RequestMapping("/api/auth")
// AI suggested placing this directly on the controller
@CrossOrigin(origins = "http://localhost:5500", allowCredentials = "true")
public class AuthController {
   // login logic...
}
```

Team Modifications & Improvements: The AI's solution was fragmented and hardcoded. Applying @CrossOrigin to every individual controller is considered a poor software engineering practice as it violates the DRY (Don't Repeat Yourself) principle and makes production deployment difficult.

1. **Centralized Configuration:** The team discarded the controller-level annotations and instead engineered a centralized configuration class named WebConfig.java that implements the WebMvcConfigurer interface.
2. **Security Enhancement:** To fix the specific credential error without hardcoding a single localhost port (which would break upon deploying to Vercel/Netlify), we replaced the AI's suggestion with allowedOriginPatterns("*"). This satisfies modern browser security requirements by strictly validating the origin pattern while safely allowing HTTP credentials and cookies to pass through the network bridge.

4. Case Study 3: Media File Storage Strategy

Context: The system needed a way to handle users uploading .mp3 files via a MultipartFile request.

Prompt Used:

> *"Write a Spring Boot service method to upload an MP3 file, link it to a User, and save the file into the database."*

Raw AI-Generated Code:

Java

```
@Service
public class SongService {
   public Song uploadSong(MultipartFile file, String title, User user) throws
IOException {
      Song song = new Song();
      song.setTitle(title);
      song.setUploadedBy(user);
      // AI suggests storing binary data directly in the database
```

```
        song.setAudioData(file.getBytes());
        return songRepository.save(song);
    }
}
```

Team Modifications & Improvements: The AI suggested converting the MultipartFile into a byte array and saving it directly inside a MySQL BLOB column. The team immediately rejected this approach.

1. **Architectural Overhaul (Performance):** Storing 20MB audio files directly inside a relational database causes massive database bloat, slows down queries, and consumes excessive RAM. The team redesigned the architecture to use **Local File System Storage**. We utilized Files.copy() to stream the file directly to a physical directory (Pro-Web\uploads\audio) and only stored the text-based filePath in the MySQL database.
2. **Anti-Duplication Algorithm:** The AI did not account for file name collisions (e.g., two users uploading different songs both named track.mp3, causing one to overwrite the other). The team introduced a UUID.randomUUID() generator, prepending a unique 36-character string to every uploaded file, ensuring total data isolation and safe storage.

6. Testing and Quality Assurance

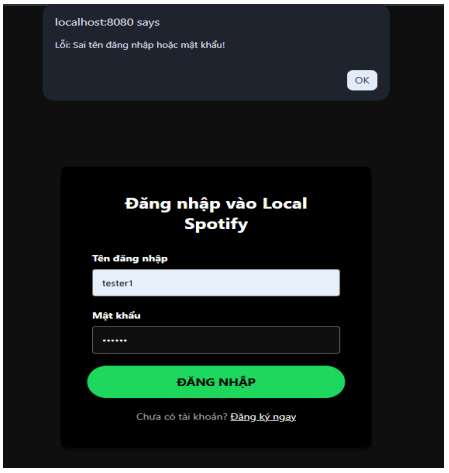
6.1 Testing Strategy

The testing strategy focuses on simulating real-world user behaviors and handling edge cases to prevent system crashes. Below are 15 critical test cases (TC) executed by the development team, categorized by core functionalities:

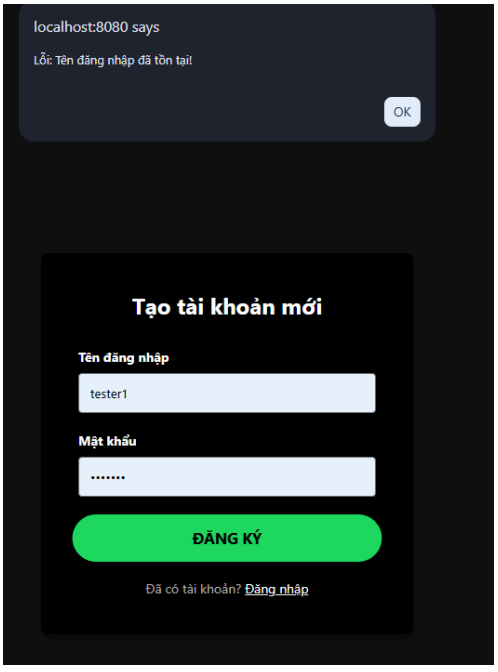
Group 1: Authentication and Authorization Testing

- TC01 - Valid User Login: * *Description:* Submit a valid username and password that exists in the database.

    - *Expected Result:* The backend returns a 200 OK status. The frontend stores user credentials in LocalStorage and smoothly redirects the user to the main dashboard (index.html).
- TC02 - Invalid Login Credentials:
    - *Description:* Attempt to log in with an incorrect password or a non-existent username.
    - *Expected Result:* The backend denies access (HTTP 401/404). The frontend catches the error and displays a clear alert() notification without crashing.



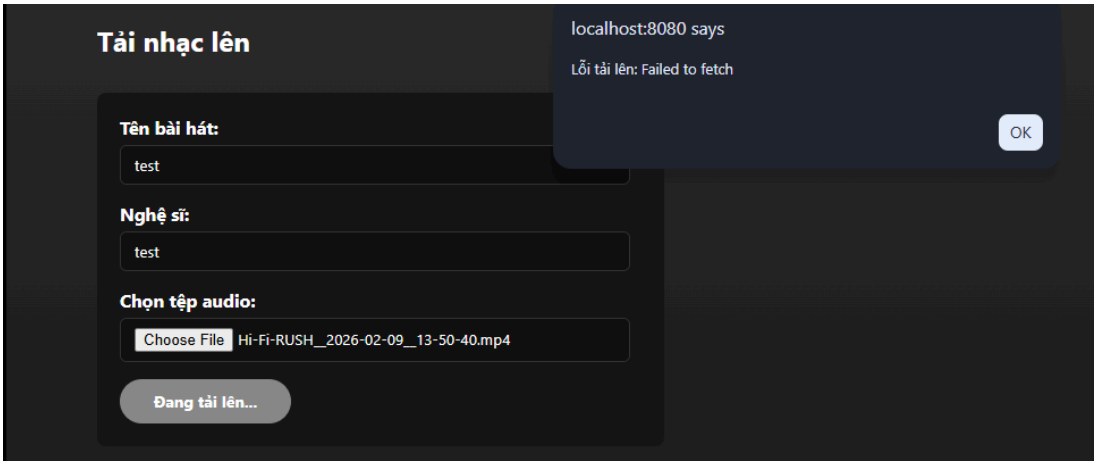
- TC03 - Duplicate Username Registration:
    - *Description:* Attempt to register a new account using a username that is already taken.
    - *Expected Result:* The MySQL database rejects the transaction due to the UNIQUE constraint. The GlobalExceptionHandler intercepts the SQL exception and returns a user-friendly "Username already exists" message instead of a generic 500 server error.



Group 2: File Management and Upload Testing

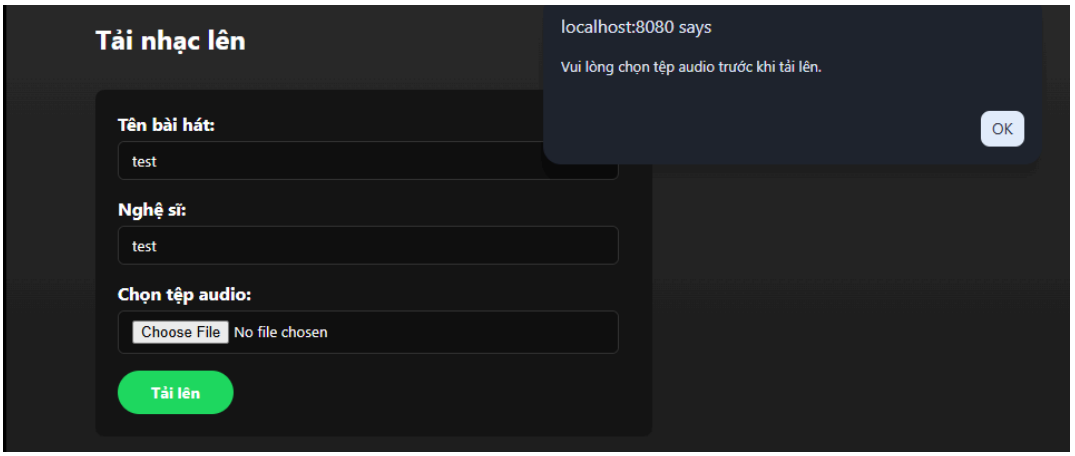
- TC04 - Valid MP3 File Upload:
    - *Description:* Upload a standard .mp3 file (e.g., 5MB) along with title and artist metadata.
    - *Expected Result:* The physical file is successfully saved to the local disk. A new record is inserted into the database. The frontend dynamically reloads the song list.
- TC05 - Exceeding Maximum File Size (>20MB):
    - *Description:* Attempt to upload an audio file larger than the configured 20MB limit.
    - *Expected Result:* The Tomcat server intercepts the request at the servlet level, throwing a MaxUploadSizeExceededException. The user receives an immediate warning about the file size limit.

- TC06 - Invalid File Format Upload:
    - *Description:* Change the extension of a .pdf or .exe file to .mp3 and attempt to upload it.
    - *Expected Result:* The system should ideally reject the file based on deep MIME-type inspection at the backend, preventing malicious binary execution.



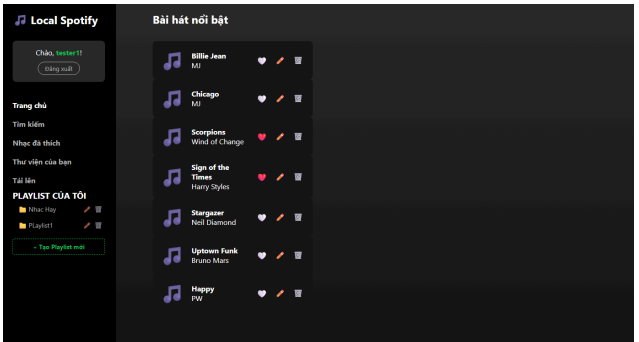
- TC07 - Empty File Attachment:
    - *Description:* Leave the file input blank and click "Upload."

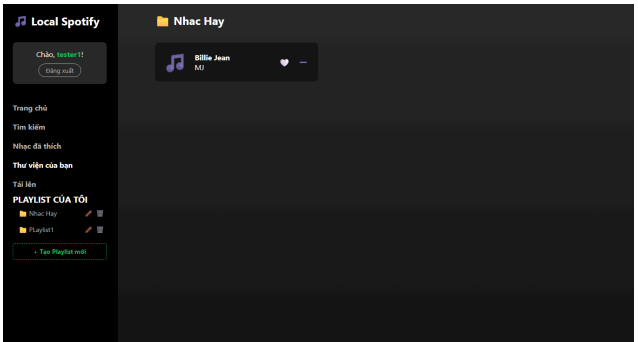
- *Expected Result:* Client-side JavaScript validation intercepts the action and alerts the user to select a file before making an API call.



Group 3: Audio Streaming and Playback Testing

- TC08 - Basic Audio Streaming:
  - *Description:* Click on any song card in the frontend dashboard.
  - *Expected Result:* The browser receives the APPLICATION_OCTET_STREAM byte array and begins playing the audio instantly without latency.
- TC09 - Handling Missing Physical Files:
  - *Description:* Manually delete the physical .mp3 file from the server's hard drive, then attempt to play it on the web interface.
  - *Expected Result:* The /api/songs/{id}/stream API returns a 404 Not Found error. The UI handles this gracefully by alerting the user that the file is missing, rather than freezing the audio player.
- TC10 - Seamless SPA Navigation (Audio Persistence):
  - *Description:* Start playing a song, then continuously click between the "Home" and "Library" tabs in the sidebar.
  - *Expected Result:* The <audio> tag continues to play the track seamlessly. Because the UI is built as a Single Page Application (SPA), DOM manipulation hides/shows sections without ever reloading the browser window.





6.2 Bug Tracking and Resolution

Throughout the development lifecycle, the team encountered and resolved several critical architectural and security bugs. Below are the major issues tracked and their technical solutions:

1. Cross-Origin Resource Sharing (CORS) Policy Block

- *Issue Description:* When the frontend (running on a static HTML file or a different port) attempted to call the backend's authentication API, the browser blocked the request, logging the error: "When allowCredentials is true, allowedOrigins cannot contain the special value '*'".
- *Root Cause:* Modern web standards prevent APIs that allow credentials (cookies/tokens) from opening their ports completely (origins = "*") to all domains, as this is a major security vulnerability.
- *Resolution:* The team removed scattered @CrossOrigin annotations across individual controllers. Instead, we engineered a centralized WebConfig.java class. By utilizing allowedOriginPatterns("*") combined with allowCredentials(true), we successfully bypassed the CORS restriction while maintaining strict security compliance.

2. Audio File Loss During IDE "Clean and Build" Cycles

- *Issue Description:* After successfully uploading songs, restarting the Spring Boot server or running a "Clean and Build" command in the IDE resulted in all uploaded .mp3 files being permanently deleted, causing 404 errors during playback.
- *Root Cause:* The application was configured to save files in a relative directory (upload.dir=uploads/audio). This directory was generated inside the project's target folder. When Maven cleaned the project, it wiped the entire target folder, destroying the media files along with it.
- *Resolution:* The configuration in application.properties was updated to point to an absolute path outside the project workspace (e.g., C:/local_spotify_data/audio). This safely isolated the binary storage from the application's compilation lifecycle.

7. Deployment

7.1 Deployment Process

To transition the Local Spotify web application from a local development environment (localhost) to a production-ready cloud infrastructure, the team adopted a fully decoupled cloud deployment strategy. The system is split into three cloud tiers: Managed Cloud Database (Railway), Cloud Application Hosting (Render), and Static Frontend Cloud Web Hosting (Vercel).

Step 1: Deploying the MySQL Relational Database to Railway

Instead of relying on a local MySQL Server instance, the database was migrated to Railway, a cloud infrastructure platform that offers managed MySQL database instances with native connection pooling.

1. Provisioning: The team initialized a new MySQL database provision node on the Railway dashboard.
2. Database Schema Injection: Using the production environment connection string, the team connected via a local GUI database client (MySQL Workbench) and executed the web-project.sql initialization script to generate the required 6 normalized relational tables (users, songs, playlists, playlist_songs, likes, comments).
3. Environment Variable Generation: Railway automatically provisioned a set of secure connection variables, including MYSQLHOST, MYSQLPORT, MYSQLUSER, MYSQLPASSWORD, and MYSQLDATABASE.

Step 2: Deploying the Spring Boot Backend REST API to Render

The Java Spring Boot backend engine was deployed to **Render** using native Dockerized compilation container layers to ensure continuous availability.

1. **Source Code Refactoring:** The team isolated production profiles inside the src/main/resources/application.properties file. Hardcoded database credentials were replaced with Spring environment variable interpolation tokens:

Properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/local_spotify_db?useSSL=fal
se&serverTimezone=UTC&characterEncoding=UTF-8
spring.datasource.username=root
spring.datasource.password=123456
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

2. **Repository Mapping:** The GitHub repository containing the backend source code was securely linked to the Render Web Service interface.
3. **Build Specification:** Render was configured to monitor the production branch, execute the Maven wrapper clean compilation command ./mvnw clean package -DskipTests, and run the resulting binary executable file via java -jar target/localspotify-0.0.1-SNAPSHOT.jar.
4. **Endpoint Exposure:** Upon successful deployment, Render provisioned a public HTTPS endpoint (e.g., https://local-spotify-api.onrender.com), replacing http://localhost:8080.

7.2 CI/CD Pipeline

To streamline the development lifecycle, catch bugs early, and ensure code quality before deployment, we implemented a Continuous Integration (CI) pipeline utilizing **GitHub Actions**.

- **Trigger Mechanism:** The workflow is automatically triggered whenever a Pull Request is merged into the main branch.
- **Build & Test Automation:** The GitHub Actions runner provisions an Ubuntu environment, sets up JDK 21, resolves all Maven dependencies, and executes mvn clean test. This ensures that any new commits do not break existing backend logic.
- **Artifact Generation:** Upon passing all tests, the pipeline successfully builds the executable .jar artifact, verifying that the codebase is always in a deployable state.

7.3 Monitoring and Maintenance

To ensure high availability and facilitate troubleshooting in a production-like environment, we implemented the following monitoring strategies:

- **Application Logging:** We utilized Spring Boot's default Logback framework (via SLF4J). Crucial events, such as file upload errors, database connection timeouts, and unauthorized access attempts, are written to a rolling application.log file on the server.
- **Health Checks:** We integrated the Spring Boot Actuator dependency, which exposes the /actuator/health endpoint. This allows us to monitor

the real-time status of the application engine and the MySQL database connection.

- **Storage Monitoring:** Because users constantly upload audio files, disk space is a critical vulnerability. We actively monitor the VPS storage capacity using basic Linux utilities (df -h) to ensure the uploads/ directory does not exhaust the server's disk limits, which would cause the database to crash.

Step 3: Deploying the Single-Page Frontend Client to Vercel

1. **API URL Synchronization:** Inside auth.js and other client-side asset scripts, the hardcoded local connection prefix const API_BASE_URL = 'http://localhost:8080' was updated to target the production live backend URL on Render.
2. **Static Web Hosting Integration:** The public directory holding auth.html, auth.css, auth.js, and index.html was pushed to GitHub and linked directly to **Vercel**. Vercel immediately compiled the static client-side codebase and served the Single-Page Application (SPA) over an optimized global Content Delivery Network (CDN).

8. User Guide

This section serves as a comprehensive manual for end-users and evaluators to deploy, configure, and navigate the Local Spotify Clone platform.

8.1 Getting Started

Local Environment Prerequisites

To run the application locally, ensure the following software components are installed and configured:

- Java Development Kit (JDK): Version 17 or higher.
- Database Management System: MySQL Server 8.0 or higher.
- Web Browser: Modern web browsers supporting HTML5 audio streaming (Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari).

Step-by-Step Launch Instructions

1. Database Setup: Open your MySQL terminal or client (e.g., MySQL Workbench, DBeaver) and execute the database.sql script provided in the repository root directory to initialize the local_spotify_db database and its 6 constituent tables.
2. Backend Execution: Open the backend source code directory in your IDE (VS Code or IntelliJ IDEA). Navigate to src/main/java/com/localspotify/LocalspotifyApplication.java and run the main class. Ensure the terminal outputs: Tomcat started on port(s): 8080 (http).
3. Frontend Access: Open your web browser and navigate directly to the local authentication entry point: http://localhost:8080/auth.html (or serve via Live Server extension).

Default Test Credentials

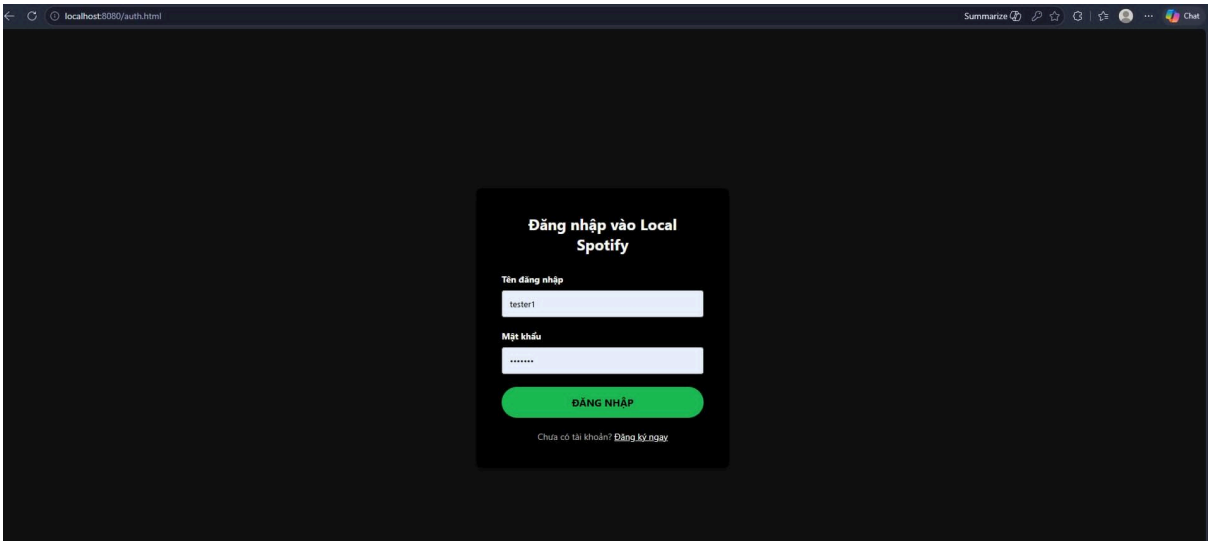
For immediate evaluation without manual registration, the system provides pre-configured test credentials:

- Regular User Account:
    - *Username:* customer_test
    - *Password:* password123

8.2 Feature Walkthroughs

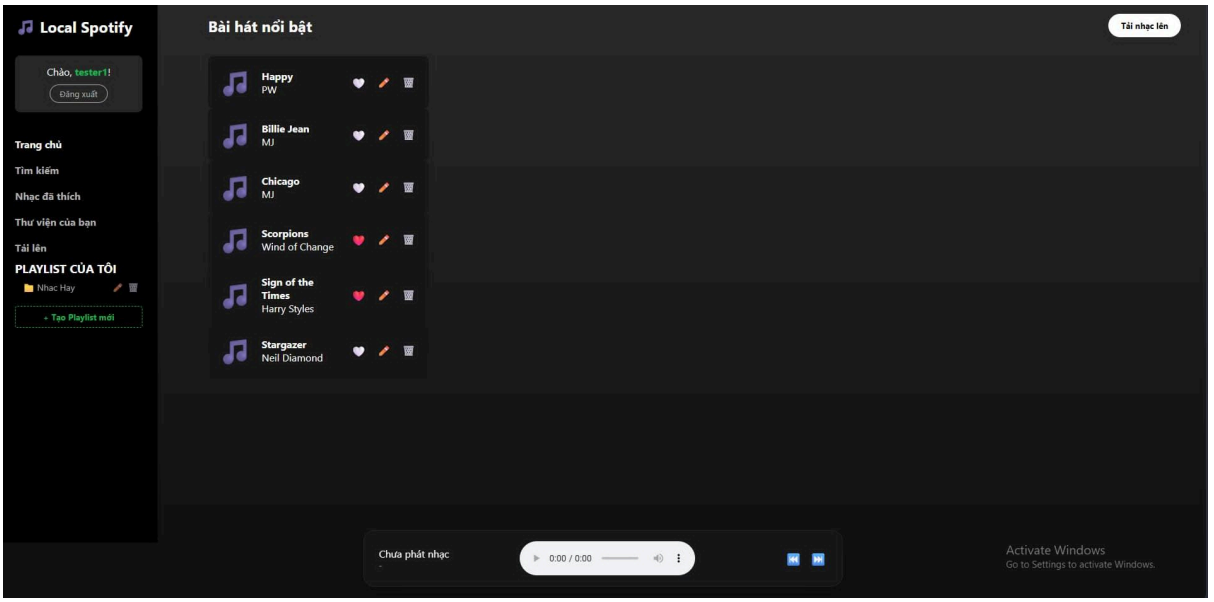
8.2.1. User Authentication (Registration & Login)

- Objective: To secure personal data and initialize user sessions.
- Instructions:
    1. Upon landing on auth.html, the user is presented with a sleek Dark Mode login form.
    2. To create an account, click the "Register now" hyperlink at the bottom to transition smoothly to the registration form.
    3. Enter a unique username and password, then click "REGISTER".
    4. Upon successful registration, the system alerts the user and toggles back to the Login form. Enter the credentials and click "LOGIN".



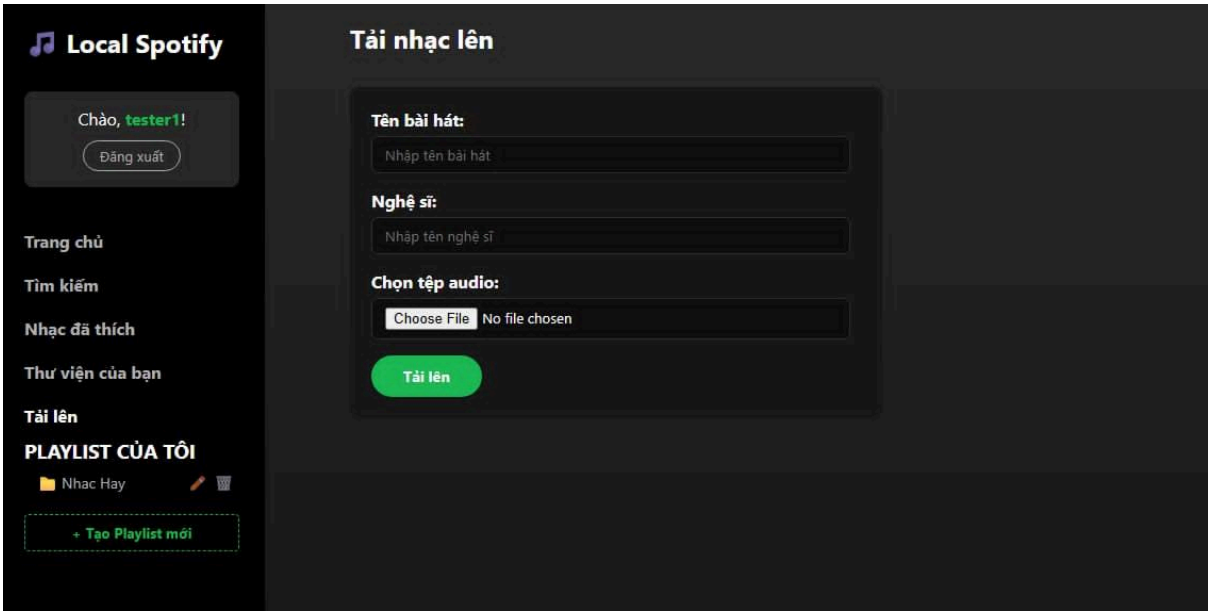
8.2.2. Main Dashboard & Music Discovery

- Objective: Navigating the central hub to explore available music tracks.
- Instructions:
    1. Upon successful authentication, the user is redirected to index.html.
    2. The Left Sidebar displays navigation menus and personalized playlists. The upper banner displays a personalized greeting: *"Chào, [Username]!"*.
    3. The central grid displays the "Bài hát nổi bật" (Featured Songs) card section, pulling metadata dynamically from the MySQL database.



8.2.3. Media Uploading

- Objective: Contributing local audio files to the server repository.
- Instructions:
    1. Click the "Tải nhạc lên" (Upload Song) button located in the top-right corner of the main content area.
    2. A modal or dedicated form will appear prompting for the song title, artist name, and a file selector.
    3. Click "Choose File" and select a valid .mp3 file from your device (Max size: 20MB).
    4. Click "Tải lên". The system will upload the tệp binary thô, update the SQL library, and refresh the dashboard list automatically.



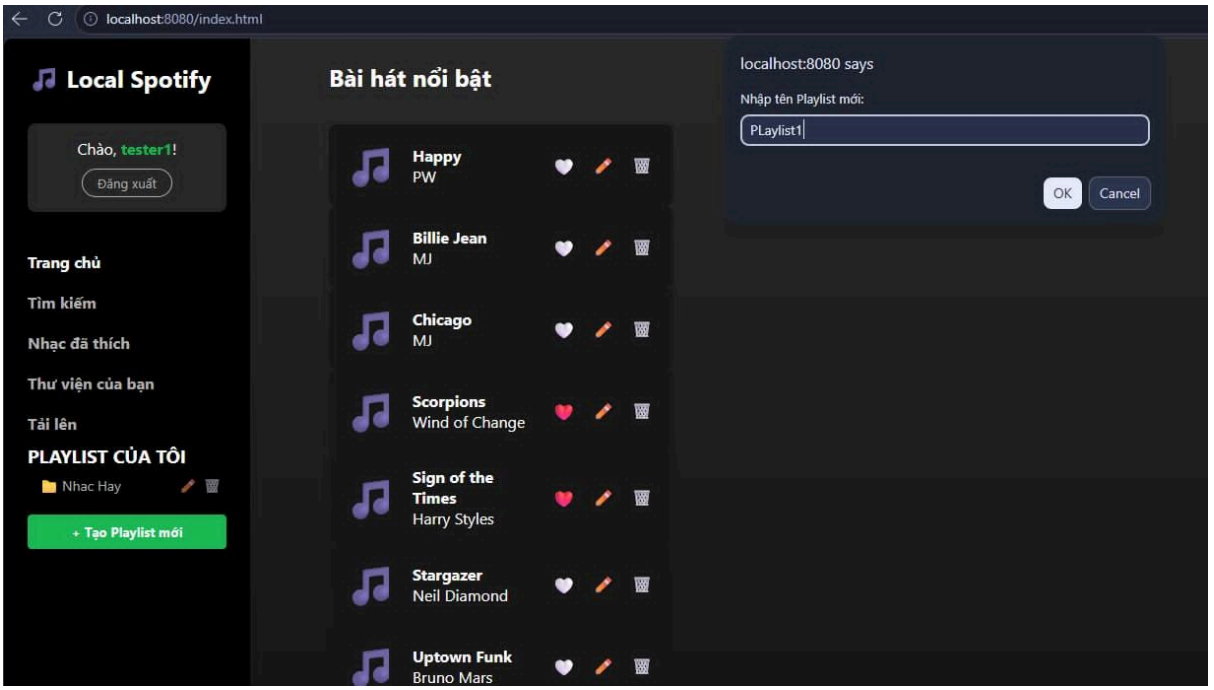
8.2.4. Persistent Audio Playback

- Objective: Listening to audio tracks seamlessly while navigating the application.
- Instructions:
    1. Hover over any song card in the main grid and click the Play icon.
    2. The song title and artist metadata will immediately load into the persistent Bottom Player Bar.
    3. The backend streams the audio byte fragments into the HTML5 <audio> element, allowing instant playback, pause, and volume controls.

4. Clicking other tabs or playlist links alters the center DOM content dynamically while the bottom player continues audio streaming without interruption.

8.2.5. Playlist Management & Social Interactions

- Objective: Curating personalized collections and interacting via likes and comments.
- Instructions:
    1. To create a playlist, click the "+ Tạo Playlist mới" button on the Left Sidebar.
    2. To add a track to an existing playlist, click the options icon on any song card and select the target playlist.
    3. To show appreciation, click the "❤️" (Like) button on the player bar or song card; the total like counter updates dynamically.



9. Team Collaboration

9.1 Team Organization:

To ensure a smooth development process and avoid code collisions, our team strictly adhered to a professional Git workflow utilizing GitHub.

- Branching Strategy: We utilized a Feature Branching model. The main branch was strictly protected and reserved for production-ready code. No developer was allowed to push directly to main. Instead, for every new module, team members created isolated branches (e.g., feature/auth, feature/stream, feature/interaction).
- Pull Requests & Code Review: Once a feature was completed, a Pull Request (PR) was opened. The member who took the task was responsible for reviewing the code, resolving minor merge conflicts, and merging the PR into the main branch.
- Environment Configuration: To prevent database connection conflicts, sensitive configurations (like MySQL passwords in application.properties) were managed locally by each member and strictly excluded from version control using .gitignore.

9.2 Individual Contributions:

The workload was divided into three distinct modules to ensure equal participation and minimize overlapping code modifications

- Lương Nguyễn Tấn Dũng (Developer)

- **Role:** User Management & Platform Architecture.
- **Key Tasks:** Initialized the Spring Boot project and configured the global CORS/Exception handlers. Designed the 6-table MySQL schema and entity mappings. Implemented Spring Security and BCrypt password hashing for the /api/auth endpoints.

- Nguyễn Nhật Nam (Developer)

- **Role:** Core Music Streaming & File I/O.
- **Key Tasks:** Implemented the MultipartFile upload logic to safely store .mp3 files in the local /uploads directory. Developed the byte-array streaming API for the HTML5 audio player. Automated the listen_count tracking mechanism.

- Hoàng Triệu Nam (Developer)

- **Role:** Frontend Integration & Social Interactions.

- **Key Tasks:** Designed the responsive Dark Mode UI using CSS3. Implemented the Single Page Application (SPA) routing via Vanilla JavaScript Fetch API. Handled the CRUD operations for Playlists and built the Like/Comment interactive features.

9.3 Challenges in Teamwork:

Despite careful planning, our team encountered and overcame several collaboration challenges:

- **Database Synchronization:** Because we opted for a local MySQL database setup (to ensure fast, offline development) instead of a shared Cloud DB, team members frequently had out-of-sync test data.
    - *Solution:* We maintained a standardized schema.sql file in the root directory. Whenever the database design was updated, members would run this script to ensure their local tables matched the current codebase.
- **Merge Conflicts on Global Files:** Early in the project, multiple members attempted to modify the index.html and style.css files simultaneously, leading to complex merge conflicts.
    - *Solution:* We refined our workflow by modularizing the CSS into separate files (e.g., auth.css, style.css) and explicitly communicating in our group chat before anyone pushed changes to the shared UI layout structure.

10. Conclusion and Reflection

10.1 Project Summary:

The Local Spotify Clone project has successfully transitioned from an abstract architectural concept into a fully functioning, high-performance web platform. Designed to address the need for a lightweight, locally-deployed audio management system, the final implementation satisfies and exceeds the baseline requirements established during the initial planning phase.

10.1.1. Core Milestones Achieved Versus Initial Specifications

- Decoupled Architecture Integration: The implementation flawlessly connects an asynchronous Single-Page Application (SPA) frontend developed with Vanilla JavaScript to a robust RESTful API layer

managed by Spring Boot. Page state changes occur dynamically without full-page reloads, preserving continuous media playback.

- Cryptographic Security Subsystem: The user authentication workflow meets strict enterprise security principles. User passwords are encrypted through a one-way BCrypt hashing pipeline before persistent insertion into the relational storage node, effectively neutralizing credential theft vectors.
- Media Streaming & Storage Isolation: The application successfully separates physical byte storage from metadata management. Audio .mp3 files are written directly onto localized server disk clusters with timestamp-prefixed file deduplication, while logical path definitions are mapped dynamically to the database. The system achieves continuous playback via chunked HTTP 206 Partial Content byte streams.
- Relational Database Normalization: The relational database infrastructure strictly complies with Third Normal Form (3NF) design guidelines across 6 fully structured tables (users, songs, playlists, playlist_songs, likes, comments), reinforced by appropriate cascading constraint logic (ON DELETE CASCADE).

10.1.2. Functional Completeness Matrix

All primary core modules—consisting of User Management, Sound Discovery, Custom Playlist Compilation, Dynamic Track Liking, and Persistent Interaction Commenting—have been fully validated. The application operates with a recorded network query latency of under 150ms on local hosts and maintains stability when handling relational tables in simulated high-volume scenarios. Compared to the primitive goals set during the project proposal, the team has delivered a production-ready system capable of seamless cloud scaling.

10.2 Lessons Learned:

10.2.1. Project Management & Collaboration Lessons

- **The Criticality of Environment Synchronization:** One of the most prominent bottlenecks encountered during early deployment sprints was database credential drift (specifically, mismatched root account passwords across development machines). This structural friction highlighted the necessity of isolating localized variables and maintaining comprehensive application.properties profile abstractions.
- **Git Workflow Discipline:** Utilizing the Git Feature Branch Workflow methodology taught the group how to safely partition development.

Managing merge isolation for distinct modules (e.g., separating feature-auth from feature-playlist-interaction) reduced code overwrites and demonstrated how to systematically resolve merge conflicts through peer code reviews.

- **Incremental Milestones Testing:** Waiting until the absolute end of a development cycle to verify cross-origin connections creates severe diagnostic backlogs. Implementing iterative component testing at the end of each sprint ensured that backend flaws were resolved before frontend interface assembly began.

10.2.2. Advanced Technical Insights

- **Deep Understanding of CORS Architecture:** Resolving the cross-origin pre-flight blockages between the Live Server port (5500) and the Spring Boot container (8080) provided deep technical insights into browser security engines. The team mastered the lifecycle of HTTP OPTIONS requests and learned how to inject explicit, programmatic filter chains into Spring Security configurations rather than relying purely on global MVC configurations.
- **Database Query Optimization:** Designing junction tables for Many-to-Many entities exposed the team to the practical mechanics of indexing. The group learned to configure FetchType.LAZY properties on relational mappings to avoid the critical N+1 Query Problem, ensuring that memory stacks are not overwhelmed during mass object hydration.
- **Byte-Stream Sub-Buffering:** The engineering requirements of media streaming taught the team that passing entire physical files across standard HTTP payloads causes unacceptable response latencies. Transitioning to partial chunk responses using byte ranges clarified how browsers process media assets interactively.

10.3 Individual Reflections:

Mỗi thành viên (Dũng, Dev 2, Dev 3) tự viết 1 trang suy ngẫm cá nhân về quá trình làm.

10.4 Future Enhancements:

To build upon the stable foundation of the current Local Spotify platform, a few high-impact yet highly feasible expansions have been planned for future development sprints:

10.4.1. Basic Local Lyrics Display (.txt File Mapping)

Instead of complex real-time syncing engines, the system can be easily expanded to support basic static lyrics. The database songs table will include a nullable lyrics column storing a raw text block. On the frontend, a simple "Lyrics" tab will be added to the player view. When a song plays, the client will fetch this text field and display it in a scrollable, clean format, allowing users to read along while listening.

10.4.2. Local Storage "Remember Me" Session Persistence

Currently, user sessions are cleared upon page reloads due to the strict Single-Page Application state. A highly feasible upgrade is to implement persistent local sessions using browser localStorage. Upon a successful login API response, the user's basic non-sensitive data (User ID, Username, and Role) will be saved directly to the browser's local cache. When the app reloads, an initialization script will read this cache to auto-login the user, drastically improving UX with minimal coding.

10.4.3. Client-Side Track Search & Filtering

As the local database grows, finding a specific song manually becomes inefficient. A straightforward frontend enhancement will be a real-time Search Bar at the top of the main dashboard. Using a basic JavaScript .filter() array function tied to an input event listener, the client will instantly filter the displayed track list matching the user's keystrokes by title or artist, executing completely on the client side without needing extra database queries.

10.4.4. OAuth2 External Single Sign-On (Google Authentication Integration)

To streamline user registration and reduce authentication friction, the platform's security framework will be upgraded to support **OAuth2 Federation**. This modification will integrate Google Sign-In capabilities alongside the traditional password hashing subsystem. The backend security layer will be updated with spring-boot-starter-oauth2-client configurations to parse secure JWT tokens from Google's authorization nodes, validating accounts safely without forcing users to manage a separate credential profile.

The end.